Laboratory Report

Course: Algorithm Analysis and Design Lab

Experiment Title: Greedy-Based Multi-Booth Job Scheduling System Using C Language

Submitted by : A. S. M. MORSHED
Student ID : MUH2425001M

1. Experiment Information

Item	Details
Course Title	Algorithm Analysis and Design Lab
Experiment No.	1
Experiment Title	Greedy-Based Customer Job Scheduling
Programming Language	C
Algorithm Technique	Greedy Algorithm
IDE	Visual Studio Code
Submission Type	Source Code + Report + Output

2. Objective of the Experiment

The objectives of this experiment are:

- To understand the concept of Greedy Algorithm.
- To implement a customer scheduling system using greedy strategy.
- To simulate a real-life multi-booth service environment.
- To calculate customer waiting times.
- To analyze booth utilization efficiency.
- To calculate average waiting time, service time, and idle time.
- To develop problem-solving skills using C programming.

3. Introduction

In many real-world systems such as banks, hospitals, ticket counters, and customer care centers, customers arrive at different times and wait for service.

Efficient scheduling is important to reduce waiting time and improve service quality.

A Greedy Algorithm makes the best immediate decision at every step.

In this experiment, customers are assigned to the booth that becomes available earliest.

This scheduling technique helps minimize waiting time and improve booth utilization.

4. Problem Description

A service center contains:

- 3 service booths
- Multiple customers

Each customer has:

1. Customer ID
2. Arrival Time
3. Service Time

The system assigns customers dynamically using a greedy strategy.

Rule: Assign the next customer to the booth that becomes available first.

5. Greedy Strategy Used

The greedy rule used in this experiment is:

Assign the next customer to the booth that becomes available earliest.

This approach helps:

- Reduce customer waiting time
- Increase booth utilization
- Reduce booth idle time

6. Algorithm

Step-by-Step Procedure

Step 1

Initialize:

- 3 booths
- Booth available times = 0

Step 2

Read customer information.

Step 3

For each customer:

- Find booth with minimum available time
- Assign customer to that booth

Step 4

Calculate:

Start Time = max(Arrival Time, Booth Available Time)

Step 5

Calculate Waiting Time:

Waiting Time = Start Time - Arrival Time

Step 6

Calculate Completion Time:

Completion Time = Start Time + Service Time

Step 7

Update booth availability.

Step 8

Repeat for all customers.

7. Source Code (C Program)

```c
#include <stdio.h>

#define BOOTHS 3
#define MAX_CUSTOMERS 100

int main()
{
    int n;

    printf("Enter number of customers: ");
    scanf("%d", &n);

    int arrival[MAX_CUSTOMERS];
    int service[MAX_CUSTOMERS];

    int boothAvailable[BOOTHS] = {0, 0, 0};
    int boothService[BOOTHS] = {0, 0, 0};
    int boothIdle[BOOTHS] = {0, 0, 0};

    int boothAssigned[MAX_CUSTOMERS];
    int start[MAX_CUSTOMERS];
    int finish[MAX_CUSTOMERS];
    int waiting[MAX_CUSTOMERS];

    // Input customer data
    printf("\nEnter Arrival Time and Service Time:\n");

    for(int i = 0; i < n; i++)
    {
        printf("Customer C%d:\n", i + 1);

        printf("Arrival Time: ");
        scanf("%d", &arrival[i]);
```

```
        printf("Service Time: ");
        scanf("%d", &service[i]);
    }

    int totalWaiting = 0;
    int totalSimulationTime = 0;

    // Greedy Scheduling
    for(int i = 0; i < n; i++)
    {
        // Find booth with minimum available time
        int selectedBooth = 0;

        for(int j = 1; j < BOOTHS; j++)
        {
            if(boothAvailable[j] < boothAvailable[selectedBooth])
            {
                selectedBooth = j;
            }
        }

        boothAssigned[i] = selectedBooth;

        // Calculate idle time
        if(arrival[i] > boothAvailable[selectedBooth])
        {
            boothIdle[selectedBooth] +=
                arrival[i] - boothAvailable[selectedBooth];
        }

        // Start time
        if(arrival[i] > boothAvailable[selectedBooth])
        {
            start[i] = arrival[i];
        }
        else
        {
            start[i] = boothAvailable[selectedBooth];
        }
```

```
        // Waiting time
        waiting[i] = start[i] - arrival[i];

        // Finish time
        finish[i] = start[i] + service[i];

        // Update booth availability
        boothAvailable[selectedBooth] = finish[i];

        // Update service time
        boothService[selectedBooth] += service[i];

        totalWaiting += waiting[i];

        if(finish[i] > totalSimulationTime)
        {
            totalSimulationTime = finish[i];
        }
    }

    // Customer Table
    printf("\n\nCustomer Scheduling Table\n");
    printf("-----------------------------------------------------
--------\n");

printf("Customer\tArrival\tService\tBooth\tStart\tFinish\tWaiting
\n");
    printf("-----------------------------------------------------
--------\n");

    for(int i = 0; i < n; i++)
    {
        printf("C%d\t\t%d\t%d\tB%d\t%d\t%d\t%d\n",
               i + 1,
               arrival[i],
               service[i],
               boothAssigned[i] + 1,
               start[i],
```

```
                finish[i],
                waiting[i]);
    }

    // Booth Statistics
    printf("\n\nBooth Statistics\n");
    printf("-----------------------------------\n");
    printf("Booth\tService Time\tIdle Time\n");
    printf("-----------------------------------\n");

    for(int i = 0; i < BOOTHS; i++)
    {
        printf("Booth %d\t\t%d\t\t%d\n",
                i + 1,
                boothService[i],
                boothIdle[i]);
    }

    // Overall Statistics
    float averageWaiting =
        (float) totalWaiting / n;

    printf("\nOverall Statistics\n");
    printf("-----------------------------------\n");

    printf("Average Waiting Time = %.2f\n",
           averageWaiting);

    printf("Total Simulation Time = %d\n",
           totalSimulationTime);

    return 0;
}
```

8. Sample Input

```
Enter number of customers: 7

Enter Arrival Time and Service Time:
Customer C1:
Arrival Time: 1
Service Time: 4
Customer C2:
Arrival Time: 3
Service Time: 5
Customer C3:
Arrival Time: 4
Service Time: 2
Customer C4:
Arrival Time: 4
Service Time: 3
Customer C5:
Arrival Time: 8
Service Time: 6
Customer C6:
Arrival Time: 9
Service Time: 2
Customer C7:
Arrival Time: 10
Service Time: 5
```

9. Sample Output

```
Customer Scheduling Table
------------------------------------------------------------
Customer        Arrival Service Booth   Start   Finish  Waiting
------------------------------------------------------------
C1              1       4       B1      1       5       0
C2              3       5       B2      3       8       0
C3              4       2       B3      4       6       0
C4              4       3       B1      5       8       1
C5              8       6       B3      8       14      0
C6              9       2       B1      9       11      0
C7              10      5       B2      10      15      0


Booth Statistics
-------------------------------------
Booth   Service Time    Idle Time
-------------------------------------
Booth 1         9               2
Booth 2         10              5
Booth 3         8               6

Overall Statistics
-------------------------------------
Average Waiting Time = 0.14
Total Simulation Time = 15
```

10. Complexity Analysis

Time Complexity

If:

- n = number of customers
- b = number of booths

Then:

Time Complexity = $O(n \times b)$

Since booths are fixed (3 booths):

Time Complexity = $O(n)$

Space Complexity

Arrays are used for storing customer information.

Space Complexity = O(n)

11. Expected Learning Outcomes

After completing this experiment, students will be able to:

- Understand greedy scheduling techniques
- Simulate real-life service systems
- Analyze scheduling performance
- Implement scheduling algorithms in C
- Calculate waiting and idle times
- Compare booth utilization efficiency

12. Conclusion

This experiment successfully demonstrated the implementation of a greedy-based customer scheduling system using the C programming language.

Customers were assigned to the earliest available booth, which minimized waiting time and improved booth utilization.

The experiment also helped understand important concepts such as:

- Greedy algorithms
- Scheduling systems
- Resource allocation
- Waiting time calculation
- Performance analysis

These concepts are widely used in operating systems, cloud computing, banking software, and customer support systems.